

RGPV Exam Insights — Project Specification

Workshop: Agentic Coding with Claude Code **Host:** Patel College of Science & Technology (PCST), Indore **Audience:** CS/IT students, mixed coding levels **Duration:** ~4 hours, team-based (3–4 students per team) **Tool:** Claude Code **Deliverable:** A deployed web tool that searches *individual RGPV exam questions* (not just papers) across years, with frequency analysis.

1. Problem Statement

RGPV publishes previous-year question papers as PDFs on rgpvonline.com. The site lets students download papers by branch/semester/year, but offers no way to:

- Search the **content** of questions (only filenames)
- Find every time a specific topic has been asked across years
- See which topics repeat most often per subject
- Filter questions by unit, marks, or difficulty

During exam prep, the question a student actually has is *"Has RGPV ever asked about BCNF? How often? What's the typical phrasing?"* No existing tool answers this.

This project builds that tool.

2. Goal

Build a working web application that:

1. Ingests RGPV B.Tech question paper PDFs
2. Extracts and parses **individual questions** with metadata
3. Stores them in a searchable database
4. Provides a clean web UI for keyword + filter search
5. Shows topic frequency analysis per subject

Scope for the workshop: One branch (CSE), one semester (Semester 5), five subjects, roughly 8 years of papers (~80 PDFs).

3. Non-Goals

To keep scope tight, the following are explicitly **out of scope**:

- AI/LLM-based question understanding or answer generation
- OCR on scanned/handwritten papers (only text-based PDFs)

- User accounts, authentication, bookmarks
 - Mobile app
 - Coverage of all branches/semesters (extension after workshop)
 - Solution generation or answer keys
-

4. Users & Use Cases

Primary user: B.Tech CSE student at PCST, in exam-prep mode.

Use cases:

1. *"What has RGPV asked about transactions in DBMS over the last 5 years?"* → Filter by subject = DBMS, keyword = "transaction"
 2. *"What are the most repeated topics in Theory of Computation?"* → Frequency view for TOC
 3. *"Show me all 7-mark questions from Unit 3 of Computer Networks."* → Combined filter: subject, unit, marks
 4. *"Compare how the same topic was asked in 2019 vs 2024."* → Chronological view of a keyword
-

5. Functional Requirements

5.1 Data Ingestion

- Download PDFs from rgpvonline.com for the chosen branch/semester/subject set
- Store PDFs locally in a structured folder: `data/raw/<subject>/<year>-<session>.pdf`
- Idempotent — re-running ingestion should not duplicate files

5.2 Text Extraction

- Extract text from each PDF using PyMuPDF (`fitz`) or pdfplumber
- Preserve question boundaries and basic formatting
- Handle multi-column layouts where present

5.3 Question Parsing

Each RGPV B.Tech paper follows a predictable structure:

- 5 Units
- 2 questions per unit (Q1/Q2 of unit, often with sub-parts a/b/c)
- Each question has a marks allocation

The parser must extract:

- `unit_number` (1-5)
- `question_number` (e.g., "Q1(a)", "Q2(b)")
- `marks` (integer)
- `question_text` (cleaned)

Acceptance: $\geq 80\%$ of questions in test set correctly parsed. Parser must log failures, not crash on them.

5.4 Storage

SQLite database with the following schema (minimum):

```
sql

CREATE TABLE papers (
  id INTEGER PRIMARY KEY,
  subject_code TEXT NOT NULL,
  subject_name TEXT NOT NULL,
  year INTEGER NOT NULL,
  session TEXT NOT NULL,      -- 'JUN' or 'DEC'
  branch TEXT NOT NULL,
  semester INTEGER NOT NULL,
  source_pdf_path TEXT,
  source_pdf_url TEXT
);

CREATE TABLE questions (
  id INTEGER PRIMARY KEY,
  paper_id INTEGER REFERENCES papers(id),
  unit_number INTEGER,
  question_number TEXT,
  marks INTEGER,
  question_text TEXT NOT NULL
);

-- Full-text search index
CREATE VIRTUAL TABLE questions_fts USING fts5(
  question_text,
  content='questions',
  content_rowid='id'
);
```

5.5 Search & Filter API

A backend (FastAPI or Flask) exposing:

- `GET /subjects` — list available subjects

- `GET /search?q=<keyword>&subject=<code>&unit=<n>&year_from=<y>&year_to=<y>&marks=<m>` — search questions
- `GET /frequency?subject=<code>` — top keywords/phrases for a subject
- `GET /question/<id>` — single question detail with paper context

5.6 Frontend

A clean web UI with:

- Subject selector (dropdown)
- Keyword search box
- Filters: unit, year range, marks
- Results: question text, year, marks, link to original PDF
- Frequency view: bar chart or word cloud of repeated topics per subject

Stack: Streamlit (faster for workshop) **or** plain HTML + vanilla JS calling the API. Choose based on team comfort.

5.7 Deployment

Each team deploys to a free tier:

- Streamlit Cloud (if Streamlit)
- Render / Railway / Vercel (if API + frontend)

Output: a public URL.

6. Non-Functional Requirements

- **Performance:** search returns in <1 second for ~80 papers / ~1,000 questions
 - **Reliability:** parser logs and skips bad questions instead of crashing
 - **Portability:** runs on Windows/Mac/Linux with Python 3.10+
 - **Code quality:** all code reviewed by humans, not blindly accepted from Claude Code
 - **Reproducibility:** `requirements.txt` + a `README.md` with setup steps
-

7. Recommended Tech Stack

Layer	Choice	Why
Language	Python 3.10+	Best library support for PDF + workshop familiarity

Layer	Choice	Why
PDF extraction	PyMuPDF (<code>pymupdf</code>)	Fast, accurate for text PDFs
Database	SQLite + FTS5	Zero-config, full-text search built in
Backend	FastAPI or Streamlit	FastAPI if separating UI; Streamlit if one-shot demo
Frontend	Streamlit or plain HTML/JS	Match team skill level
Scraper	<code>requests</code> + <code>beautifulsoup4</code>	Standard, simple
Charts	<code>plotly</code> or <code>matplotlib</code>	Frequency visualization
Deployment	Streamlit Cloud / Render	Free tier, fast deploy

8. Project Structure

```

rgpv-exam-insights/
├── data/
│   ├── raw/                # downloaded PDFs
│   └── processed/          # extracted text (optional cache)
├── src/
│   ├── scraper.py          # downloads PDFs from rgpvonline.com
│   ├── extractor.py        # PDF → raw text
│   ├── parser.py           # raw text → structured questions
│   ├── db.py               # SQLite schema + helpers
│   ├── search.py           # query logic
│   └── app.py              # Streamlit/FastAPI entry
├── tests/
│   └── test_parser.py       # parser tests on sample papers
├── requirements.txt
├── README.md
└── SPEC.md                  # this file

```

9. Workshop Phases

Phase 1 — Setup & Ingestion (45 min)

- Install Claude Code, set up the repo

- Direct Claude Code to write `scraper.py` for one subject
- Verify PDFs land in `data/raw/`
- **Lesson:** Always look at the actual website structure before prompting

Phase 2 — Extraction & Parsing (75 min)

- Write `extractor.py` (straightforward — PyMuPDF one-liner)
- Write `parser.py` — the hard part
- Test on 3 papers, then run on all, find breakages, iterate
- **Lesson:** Claude Code's first parser works on the sample you showed it and breaks on everything else. Real engineering happens in the iteration.

Phase 3 — Storage & Search (60 min)

- Initialize SQLite schema
- Load parsed questions
- Build search function using FTS5
- **Lesson:** FTS5 makes search trivial — knowing the right tool matters more than writing more code

Phase 4 — UI & Frequency Analysis + Deploy (60 min)

- Build Streamlit UI (or API + simple frontend)
 - Add frequency analysis: top keywords per subject (use `scikit-learn` TF-IDF or simple word counts)
 - Deploy to free tier
 - Demo
-

10. Acceptance Criteria

The workshop project is considered **complete** when:

1. ☒ At least 30 PDFs ingested for the chosen subjects
 2. ☒ $\geq 80\%$ of questions correctly parsed (manual spot-check on 20 random questions)
 3. ☒ Search works on keyword + at least 2 filters
 4. ☒ Frequency view shows top 10 repeated terms per subject
 5. ☒ App is deployed and accessible at a public URL
 6. ☒ `README.md` documents how to run locally
-

11. Stretch Goals (for advanced teams)

- **Topic tagging:** classify questions by unit topic using keyword rules (not ML)
 - **Similar question detection:** find questions that are near-duplicates across years (TF-IDF + cosine similarity)
 - **Difficulty heuristic:** classify questions as short/medium/long-answer based on marks + word count
 - **Coverage gap finder:** show which units have fewer questions in recent years
 - **Multi-subject support:** extend beyond Sem 5 CSE to other branches
 - **Export:** let students download filtered question sets as PDF for offline prep
-

12. Risks & Mitigations

Risk	Likelihood	Mitigation
rgpvonline.com is slow/blocks during workshop	Medium	Pre-download all PDFs the day before; ship as zip
PDF parser fails on older papers (pre-2015)	High	Scope to 2017+ papers; older papers as stretch
Some PDFs are scanned images	Low (for recent papers)	Skip non-text PDFs; log them
Team gets stuck on regex hell	Medium	Provide a starter parser that handles 50% of cases; teams improve from there
Deployment platform signup eats time	Medium	Pre-create accounts; provide deployment cheat sheet

13. Teaching Outcomes

What students should walk out understanding:

1. **Specs first, code second.** This document exists because the spec drives the build, not the other way around.
2. **Claude Code is a fast typist, not an architect.** Teams that hand it concrete data, file paths, and small tasks succeed. Teams that ask it to "build a question search app" produce slop.
3. **Real data breaks pretty code.** Every team's first parser will fail. Debugging that failure is the actual skill being trained.
4. **Pick the right tool, write less code.** SQLite FTS5 vs. building a search engine. PyMuPDF vs. parsing PDFs by hand.

5. **Ship something real.** A deployed URL with real RGPV data, usable by the next batch of PCST students, is the measure of success.
-

14. After the Workshop

If the deployed tool gets adoption (50+ unique users in the first month), the strongest team's repo becomes a PCST-branded open-source project. The workshop is then not a one-day event but the start of a maintained student utility.

End of specification.